Projeto de Programação Multiparadigma [v1.0]
2025/2026

Enunciado

Um projeto de programação multiparadigma tira partido das vantagens de diferentes paradigmas de programação de forma a produzir uma solução mais adequada (p.e., desempenho, leitura/interpretação, manutenção). Uma divisão clássica consiste na utilização do paradigma de programação orientado aos eventos para o desenvolvimento da **camada de apresentação** (*User Interface*) e outro(s) paradigma(s) (funcional, orientado aos objetos, imperativo, lógico) para o desenvolvimento da **camada de negócio**. A **camada de dados** completa uma típica arquitetura de 3 camadas.

Aviso: O trabalho entregue deve corresponder ao esforço individual de cada grupo, e em nenhum caso deve ser copiado código que será entregue. A deteção de código copiado será realizada por software especializado bastante sofisticado. Casos de plágio óbvio serão penalizados com a anulação do projeto, o que implica a reprovação à Unidade Curricular (UC). Adicionalmente, a situação será reportada à Comissão Pedagógica da ISTA/Conselho Pedagógico do ISCTE-IUL. Serão penalizados da mesma forma tanto os alunos que fornecem código (se for o caso) como os que copiam código.

Grupos de Trabalho

O trabalho deverá ser realizado por grupos de **3 elementos** (exceções necessitam de aceitação por parte do coordenador da UC).

Datas

- Entrega nº1: até 13 de abril (via Moodle) contendo as tarefas T1, T2, T3, T4 da 1ª parte.
- Discussão nº1: 22 a 24 de abril (no horário habitual das aulas)
- Entrega nº2: até 18 de maio (via Moodle) contendo a 1ª parte (melhorada) e a 2ª parte
- Discussão nº2: durante uma das aulas (podem ser necessárias *slots* adicionais) da última semana de aulas – 19 a 22 de maio

Introdução

O objetivo deste projeto é o desenvolvimento de competências de programação que permitam desenvolver, em colaboração, aplicações multiparadigma interativas de média escala. O projeto divide-se em duas partes obrigatórias. A primeira parte corresponde ao desenvolvimento da camada de negócio utilizando, obrigatoriamente, a linguagem de programação **Scala** seguindo os princípios de programação funcional pura (exceções serão explicitamente indicadas). A segunda parte corresponde ao desenvolvimento da camada de apresentação (interativa) que deverá ser desenvolvida utilizando **JavaFX**.

Parte 1: Camada de Negócio

Objetivos: Desenvolver um conjunto de métodos, utilizando o paradigma de programação funcional, capazes de garantir os objetivos funcionais da solução proposta.

Parte 2: Camada de Apresentação Interativa

Objetivos: Desenvolver uma *text-based User Interface* (TUI) e uma *Graphical User Interface* (GUI) que permitam uma interação fácil e intuitiva com a aplicação.

Requisitos

O projeto deverá, adicionalmente, satisfazer os seguintes requisitos:

Funcionais

1. Permitir executar com sucesso as várias tarefas/métodos (ver secção Tarefas) **mantendo as assinaturas indicadas**;
2. Manter estado entre execuções, i.e., guardar o estado de jogos não terminados para execuções futuras.

Não funcionais

1. Apresentar ambas interfaces: gráfica (GUI) e textual (TUI);
2. Os utilizadores deverão conseguir interagir corretamente (com o menor número de erros cometidos) e com facilidade (equilíbrio adequado entre quantidade de informação apresentada e número de ações necessárias para realizar uma tarefa) sem necessidade de treino.

Tema – Jogo *Kōnane*

Neste trabalho prático ocupar-nos-emos com a implementação do jogo de tabuleiro *Kōnane*[1]. O tabuleiro retangular, de tamanho variável, é inicializado com pedras em todas as posições dispostas com um padrão alternado e, removendo do centro ou de um canto uma pedra preta e uma pedra branca ortogonalmente adjacentes (ver exemplo na Figura 1). Em termos de objectivo, quando um dos jogadores não puder efetuar uma jogada válida perde o jogo.
Cada jogador joga, alternadamente, para capturar as pedras do adversário. Todas as jogadas devem ser movimentos de captura. Um jogador captura uma pedra inimiga adjacente, saltando sobre ela (em qualquer das seis direções) com a sua própria pedra. A pedra do jogador salta sobre a pedra inimiga adjacente e pousa num espaço vazio imediatamente a seguir. A pedra do jogador pode continuar a saltar, em qualquer direção, por cima das pedras inimigas capturando-as. O jogador pode parar de saltar por cima das pedras inimigas a qualquer momento, mas deve capturar pelo menos uma pedra inimiga por jogada (ver Figura 3). Assim que o jogador decide parar de saltar, a vez do jogador termina. Em cada jogada, apenas uma pedra pode ser usada para capturar pedras inimigas.

Tarefas

- **T1**: implementar o método `def randomMove(lstOpenCoords: List[Coord2D], rand: MyRandom):(Coord2D, MyRandom)` responsável por gerar uma coordenada aleatória para a próxima jogada a partir da lista de posições livres fornecidas - `lstOpenCoords`. A assinatura do método recebe e devolve `MyRandom` (ver material das aulas) para poder ser puro.
  O jogo deve devolver coordenadas aleatórias diferentes em diferentes execuções do programa;

- **T2**: implementar o método `def play(board:Board, player: Stone, coordFrom:Coord2D,coordTo:Coord2D,lstOpenCoords:List[Coord2D]):(Option[Board],List[Coord2D])` responsável por devolver um novo tabuleiro movendo a pedra indicada (`coordFrom`) para a coordenada fornecida (`coordTo`), se a mesma representar uma jogada válida e `None` caso contrário. Para além do tabuleiro, é devolvida a nova lista de coordenadas livres;
  Nota: deve implementar um método adicional para inicializar o `board` (ver figura 1);

- **T3**: implementar o método de ordem superior `def playRandomly(board:Board, r:MyRandom,player:Stone,lstOpenCoords:List[Coord2D], f:(List[Coord2D],MyRandom)=>(Coord2D,MyRandom)):(Option[Board],MyRandom,List[Coord2D],Option[Coord2D])` responsável por jogar de forma aleatória numa das coordenadas livres do tabuleiro. Para além do tabuleiro, é devolvida a nova lista de coordenadas livres, novo `MyRandom`, e coordenada para onde foi movimentada a pedra.

[1] https://en.wikipedia.org/wiki/Kōnane

- **T4**: representar, visualmente, as jogadas no tabuleiro na linha de comando (ver exemplo de possibilidade de representação na Figura 2)

- **T5:** implementar o método responsável por verificar se o computador ou o jogador ganhou o jogo.

- **T6**: adicionar um temporizador limite (configurável no início do jogo) para cada jogada e, permitir que seja possível após cada jogada realizar *undo*, i.e., anular a última movimentação do jogador e do computador.
  Pode utilizar o método `System.currentTimeMillis()` para obter o tempo atual;

- **T7**: desenvolver uma TUI disponibilizando as várias opções de interação (p.e., jogar, continuar ou parar de capturar pedras inimigas numa jogada, reiniciar, alterar dimensões do tabuleiro, definir o tempo máximo para cada jogada, definir o nível de dificuldade) do jogo;

- **T8**: desenvolver uma GUI (tabuleiro de dimensão fixa i.e., 6x6) com as mesmas opções que a TUI e, adicionalmente, uma interação com usabilidade mais adequada (p.e., ao selecionar uma dada coordenada com o cursor sem ter que indicar, manualmente, as coordenadas; visualizar as diferentes alternativas de jogada válidas).

Fatores de valorização:

- Utilização de conceitos de programação funcional "avançada" (p.e., *fold*, *tail recursion*);
- Adicionar níveis de dificuldade ao jogo melhorando o desempenho do computador;
- Extras.

Tipo de Dados a Utilizar Obrigatoriamente

```
type Coord2D = (Int, Int) //(row, column)

type Board = ParMap[2][Coord2D, Stone]

//Enum in Scala 3
enum Stone:
    case Black, White
```

[2] adicionar a seguinte dependência no ficheiro *build.sbt* para poder utilizar *parallel collections*:
`libraryDependencies += "org.scala-lang.modules" %% "scala-parallel-collections" % "1.2.0"`

Exemplos de Execução do Jogo Kōnane

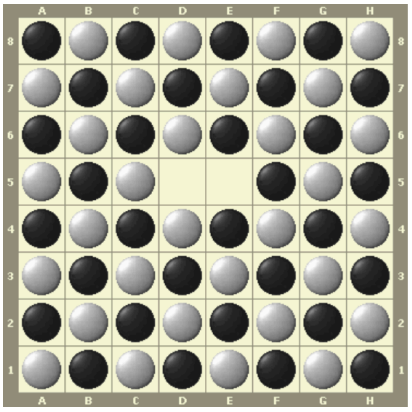

Figura 1 – Exemplo de inicialização válida de um tabuleiro 8x8.

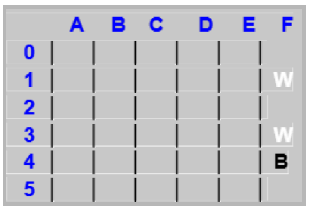

Figura 2 – Exemplo de execução parcial do jogo num tabuleiro 6x6 via TUI

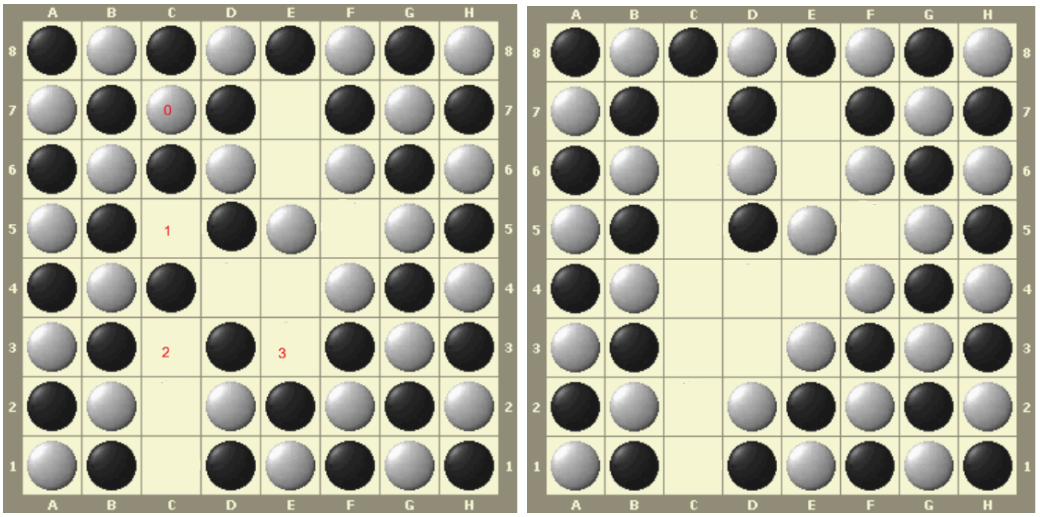

Figura 3 – Tabuleiro inicial (à esquerda da imagem) e tabuleiro resultante da captura de três pedras (jogada: 0 -> 1 -> 2 -> 3) à direita da imagem.

Avaliação

A realização de <u>ambas as partes</u> do projeto é obrigatória para obter aprovação à UC. Não existe qualquer possibilidade de obter aprovação à UC sem realizar o projeto. O projeto é composto por duas partes e realizado em grupo, no entanto, as discussões

e aferições são individuais. As classificações possíveis na aferição individual são A, B ou C, definindo a nota final individual do projeto (*nf*):

- A: *nf* igual à classificação do projeto;
- B: *nf* igual a 80% da classificação do projeto;
- C: reprovação à UC.

A **primeira parte** do projeto (com peso de **75%**) será inicialmente avaliado em termos funcionais (i.e., se os métodos produzem os resultados esperados) e se cumpre os requisitos funcionais (para ser usado numa primeira fase via TUI). A solução desenvolvida deverá, obrigatoriamente, aplicar as seguintes matérias de programação funcional introduzidos na UC:

- Recursividade (no lugar de iteratividade) e *Pattern matching*;
- Imutabilidade (exceto na GUI);
- *Pure functions* (exceto na GUI);
- Função de ordem superior;
- *Functional Object-Oriented programming*.

Ainda que a funcionalidade exigida tenha sido concretizada, a não aplicação ou aplicação inadequada das matérias mencionadas irá penalizar a classificação. A avaliação também poderá ser penalizada em função da reduzida qualidade do código (p.e., difícil manutenção, baixa eficiência, dificuldade de interpretação, duplicação, complexidade desnecessária).

A **segunda parte** do projeto (com peso de **25%**) será avaliada em termos da qualidade e usabilidade das interfaces (TUI e GUI) desenvolvidas, do cumprimento dos requisitos e da correta aplicação da matéria sobre engenharia de sistemas interativos (*Event-driven Programming*).

Apesar de não ter um peso na avaliação, a utilização de tecnologias para suporte à programação em equipa (e.g., Git, GitHub) é aconselhado.

Os alunos poderão obter feedback junto dos professores das respetivas turmas sobre o progresso do projeto, e deverão seguir as recomendações dadas.

Os projetos são classificados de acordo com os seguintes pesos:

1ª parte (75%):

- Requisitos funcionais – 45%
- Fatores de valorização – 10%
- Correta e completa aplicação da matéria de Programação Funcional – 15%
- Qualidade do código – 5%

2ª parte (25%):

- Qualidade das interfaces desenvolvidas – 20%
- Correta aplicação da matéria de sistemas interativos – 5%

A versão do projeto da entrega nº1 terá um peso de 10% na nota final e será classificado da seguinte forma[3]:

- 2 valores – as 4 tarefas solicitadas para a entrega nº1 implementadas de forma adequada;
- 1 valor – 3 das tarefas solicitadas para a entrega nº1 implementadas de forma adequada;
- 0.5 valores – 2 das tarefas solicitadas para a entrega nº1 implementadas de forma adequada;
- 0 valores – caso contrário.

As tarefas da entrega nº1 podem ser corrigidas/melhoradas na entrega nº2 para melhorar a avaliação final do projeto, no entanto, a nota da entrega nº1 ficará inalterada.

Entrega

Submissões: ambas as entregas deverão ser, obrigatoriamente, realizadas por cada grupo através do Moodle.

Dados e formatação:

- Projeto com todo o código fonte desenvolvido em formato *Zip* ("File/Export/Project to Zip File…") **contendo todos os dados necessários para poder testá-lo**. O nome do projeto deve incluir o **número do grupo e nome dos vários membros** (*NúmeroGrupo_Nome&Número1_Nome&Número2_Nome&Número3*).

  Garantir que o nome do seu projeto, tanto no IDE como no nome do ficheiro zip, contém informação que permite identificar o número do grupo e nome de cada membro;

  O código do seu projeto não pode usar caracteres especiais (á, à é, ç, etc.);

  Sugestão: verifiquem que a importação do vosso projeto numa máquina diferente ocorre com sucesso antes de efetuar a submissão;

- Possíveis comentários/considerações (ficheiro: *Readme.txt*).

O ficheiro Readme.txt deve ser adicionado à raiz do projeto.

Caso não cumpra os procedimentos em cima, o seu projeto não se conseguirá importar com facilidade, obrigando os docentes a realizar *setups* manuais que atrasam de forma muito significativa o processo de avaliação dos projetos. Como tal, os projetos entregues incorretamente (i.e., que não se consigam importar automaticamente para o *IntelliJ* a partir de um ficheiro zip) **serão penalizados com 2 valores**.

[3] podendo sofrer penalizações em função do desempenho do estudante durante a discussão nº1.

Cada membro do grupo deve submeter, individualmente, na entrega final a autoavaliação do trabalho em grupo (ficheiro: grupo_*NúmeroGrupo*.pdf) – ver tabela no anexo I. Este ficheiro deverá ser submetido através do Moodle num outro *link* de submissão que será criado para o efeito.

Discussão: a discussão é individual e todos os alunos terão de demonstrar ser capazes de fazer um trabalho com o nível de qualidade igual ao que entregaram. Ao entregar o trabalho, os alunos implicitamente afirmam que são os únicos responsáveis pelo código entregue e que todos os membros do grupo participaram de forma equilibrada na sua execução, tendo todos adquirido os conhecimentos necessários para produzir, individualmente, um trabalho equivalente.

Anexo I - Avaliação do trabalho em grupo

Classifique os membros do seu grupo (incluindo-o a si próprio) acerca dos itens seguintes numa escala até 20. Coloque o nome de cada membro no cabeçalho da respetiva coluna.

Responsabilidades	***Minha (preencher)***	***MembroA (preencher)***	***MembroB (preencher)***
Contribuiu para a sua parte do trabalho			
Assistiu a todas as reuniões calendarizadas			
Foi cooperativo			
Teve uma atitude positiva			
Foi tolerante a outros pontos de vista			
Teve espírito de grupo			
Completou todo o trabalho a si atribuído e no tempo estipulado			
Esforçou-se para produzir resultados de qualidade			
Incluiu os outros membros nas decisões tomadas e discussões			
Ajudou os outros membros do grupo			